

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

CO1 ASSESSMENT TOOL 1 – Case Study Analysis

Course Code:	CSA10 / CSA1023	Course Title:	Software Engineering
CO Assessed:	CO1	Assessment:	Assessment Tool 1 – Case Study Analysis
Weightage:	20%	Total Marks:	25
CO1 Statement:	<i>Apply advanced methodologies and agile project management techniques to manage software projects effectively</i>		
Student Name:	Beffy A Shanika	Reg. No.:	192511387
Date:	06/10/2026	Duration:	60 minutes

Code of Conduct

I Beffy A Shanika (192511387) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: Beffy _____

Annexure A – Case Study Analysis

Instructions to Students:

Each student will be allotted **one problem statement**. Analyze the assigned problem systematically by following the steps given below. Your report should demonstrate your understanding of software engineering concepts and your ability to design an appropriate software solution.

Based on your allotted problem statement, prepare a report by answering the following questions:

1. Understand the Problem Statement

- Explain the problem in your own words.
- Identify the objectives of the proposed system.
- State the scope of the problem.

2. Identify Key Stakeholders and Challenges

- Identify all the stakeholders involved in the system.
- Explain the roles and responsibilities of each stakeholder.
- Discuss the major challenges and issues faced by the stakeholders.

3. Analyze the Current Approach

- Describe the existing system or current process.
- Identify its strengths and limitations.
- Analyze the problems or gaps that need to be addressed.

4. Propose a Suitable Solution Using Software Engineering Principles

- Design a software-based solution for the given problem.
- Describe the major functional modules of the proposed system.
- Identify the functional and non-functional requirements.
- Explain how software engineering principles such as modularity, scalability, maintainability, reliability, usability, and security are incorporated into your solution.

5. Justify the Chosen Methodology/Framework

- Select an appropriate Software Development Life Cycle (SDLC) model or software development framework (e.g., Agile, Scrum, Waterfall, Spiral, DevOps).

- Justify why the selected methodology/framework is suitable for the proposed system.
 - Explain how it supports successful project development and maintenance.
- 6. Discuss Expected Outcomes and Limitations**
- Describe the expected benefits of the proposed solution.
 - Explain how the solution addresses the identified challenges.
 - Discuss potential risks, implementation challenges, and limitations.
 - Suggest possible future enhancements.

Answer:

Legacy-to-Microservices Modernization and Technical-Debt Recovery Platform

1. Understand the Problem Statement

1.1 Problem Explanation

Many organizations still depend on large legacy applications that were developed using older technologies and monolithic architectures. These applications are often difficult to modify, test, scale, and integrate with modern services. They may contain tightly coupled modules, outdated dependencies, complex code, and limited documentation.

Completely replacing such systems can be risky because important business operations depend on their existing functionality. A failure during migration could cause service interruption or loss of important business functions.

The proposed Legacy-to-Microservices Modernization and Technical-Debt Recovery Platform provides a gradual approach to modernization. It analyzes the legacy application, identifies technical debt and tightly coupled components, selects suitable components for migration, and gradually converts them into independent microservices. The platform also supports APIs, containers, automated testing, CI/CD, monitoring, and rollback.

1.2 Objectives of the Proposed System

The main objectives are:

1. Analyze legacy applications and identify technical debt.
2. Identify tightly coupled, complex, and high-risk components.
3. Select suitable components for incremental modernization.
4. Support API-first and microservices-based migration.
5. Automate regression and integration testing.
6. Use containerization for migrated services.
7. Integrate migrated services with CI/CD pipelines.
8. Monitor application performance and migration failures.
9. Provide rollback mechanisms for failed deployments.
10. Maintain documentation and track modernization progress.

1.3 Scope of the Problem

The platform covers the complete modernization process, including:

- Legacy application analysis
- Technical-debt analysis
- Dependency and coupling analysis
- Component prioritization
- API development
- Microservice migration
- Containerization
- Automated testing
- CI/CD integration
- Monitoring and logging
- Rollback and recovery
- Documentation
- Modernization progress tracking

The main focus is incremental modernization without major disruption to existing business services.

2. Identify Key Stakeholders and Challenges

2.1 Key Stakeholders

Stakeholder	Role and Responsibilities
Developers	Analyze legacy code, refactor components, and develop microservices and APIs.
Software Architects	Design the target architecture and decide suitable migration strategies.
DevOps Engineers	Manage CI/CD pipelines, containers, deployment, and automation.
QA Team	Perform regression, integration, functional, and performance testing.
System Administrators	Manage servers, infrastructure, and application environments.
Project Managers	Manage schedule, resources, cost, and migration risks.
Business Owners	Identify critical business functions and ensure business continuity.
Security Team	Ensure APIs, services, containers, and infrastructure are secure.
End Users	Use the application and depend on its availability and performance.
Management	Approve modernization plans, resources, and major decisions.

2.2 Major Challenges

Developers

- Understanding old and undocumented code.
- Working with outdated technologies.
- Managing tightly coupled modules.
- Ensuring existing functionality is not changed accidentally.

Architects

- Identifying suitable microservice boundaries.
- Selecting which components should be migrated first.
- Maintaining communication between legacy and modern systems.

DevOps Engineers

- Managing both legacy and modern environments.
- Creating new CI/CD pipelines.
- Containerizing migrated services.

QA Team

- Creating regression tests for existing behavior.
- Ensuring migrated services work correctly.
- Testing communication between different services.

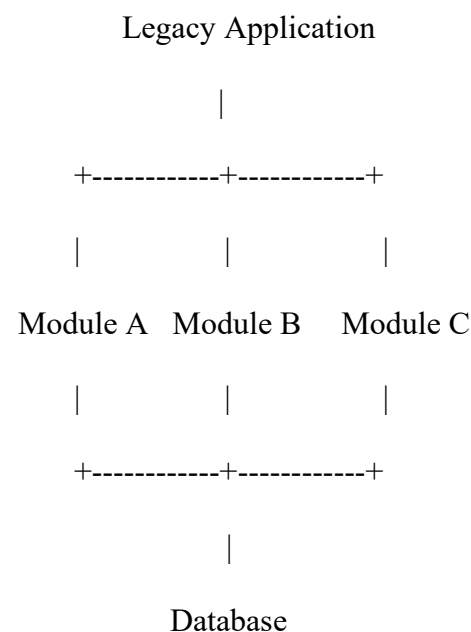
Business Owners

- Risk of service disruption.
- High migration cost.
- Dependence on critical legacy functions.

3. Analyze the Current Approach

3.1 Existing System

Most legacy applications follow a monolithic architecture, where several business functions are developed inside one large application.



3.2 Strengths of the Current Approach

The existing system has some advantages:

- Important business functions are already implemented.
- Employees are familiar with the system.
- Existing business rules and data can be retained.
- The system may provide stable and predictable functionality.
- Complete replacement is not immediately necessary.

3.3 Limitations

However, the legacy approach has several limitations:

- High technical debt.
- Tightly coupled components.
- Difficult maintenance.
- Outdated technologies and dependencies.
- Limited scalability.
- Difficult testing.
- Slow deployment.
- Poor documentation.
- High risk when modifying existing components.

3.4 Problems and Gaps

Existing Problem	Proposed Solution
High technical debt	Technical-debt analysis and refactoring
Tight coupling	Dependency and coupling analysis
Difficult scaling	Independent microservices
Outdated dependencies	Dependency modernization
Difficult testing	Automated regression testing
Manual deployment	CI/CD automation
Migration risk	Incremental migration
Poor monitoring	Centralized monitoring
Failed migration	Rollback mechanism

4. Proposed Solution Using Software Engineering Principles

4.1 Proposed Solution

The proposed platform gradually transforms selected parts of the legacy application into independent microservices. Instead of replacing the entire system, components are analyzed and migrated one by one.

Legacy Application



Application Analysis



Technical-Debt Analysis



Component Selection



API Development



Microservice Development



Containerization



Automated Testing



CI/CD Deployment



Monitoring



Rollback if Required

During migration, the legacy application and new microservices can operate together. This reduces the risk of business disruption.

4.2 Major Functional Modules

Module 1 – Legacy Application Analysis

This module analyzes source code and identifies:

- Application modules
- Dependencies
- Code complexity
- Tightly coupled components
- Outdated technologies
- High-risk components

Module 2 – Technical-Debt Analysis

This module identifies problems such as:

- Duplicate code
- Complex code
- Obsolete libraries
- Poor documentation
- Difficult-to-maintain components

The results help prioritize modernization work.

Module 3 – Component Prioritization

The system selects suitable components for migration based on:

- Business importance
- Technical debt
- Complexity
- Coupling
- Change frequency
- Migration risk

Module 4 – API and Microservice Migration

Selected components are converted into independent services using APIs. The APIs provide communication between the new services and existing application components.

Module 5 – Containerization

Migrated services are packaged into containers. This provides consistent environments and makes deployment and scaling easier.

Module 6 – Automated Testing

The platform supports:

- Unit testing
- Regression testing
- Integration testing
- API testing
- Performance testing

Regression testing ensures that modernization does not change important existing behavior.

Module 7 – CI/CD Integration

The migrated services are integrated with automated CI/CD pipelines.

Code Commit



Build



Testing



Security Check



Container Build



Deployment



Monitoring

Module 8 – Monitoring and Observability

The system monitors:

- Service availability
- Response time
- Error rate

- Resource usage
- API failures
- Deployment failures

Module 9 – Rollback and Recovery

If a new service causes problems, the platform can return to a previously stable version. This reduces the impact of failed migrations.

Module 10 – Documentation and Progress Tracking

The system records:

- Migrated components
- Remaining legacy components
- APIs
- Dependencies
- Test results
- Deployment history
- Technical-debt status
- Migration progress

4.3 Functional Requirements

The system should:

1. Analyze legacy applications.
2. Identify dependencies and technical debt.
3. Detect tightly coupled components.
4. Identify suitable migration candidates.
5. Support API development.
6. Support microservice migration.
7. Support containerization.
8. Perform automated testing.
9. Integrate with CI/CD pipelines.
10. Monitor migrated services.
11. Support rollback.
12. Maintain version control.
13. Maintain modernization documentation.
14. Generate modernization reports.

4.4 Non-Functional Requirements

Maintainability

The system should make software easier to modify and maintain.

Scalability

Individual microservices should be scalable according to their workload.

Reliability

Automated testing, monitoring, and rollback should improve deployment reliability.

Availability

Migration should minimize interruption to existing services.

Performance

The modernized system should provide acceptable response time and resource usage.

Security

APIs, containers, source code, and infrastructure should be protected from unauthorized access.

Usability

The platform should provide a simple dashboard for developers, architects, and managers.

Auditability

Migration, testing, deployment, and rollback activities should be recorded.

4.5 Software Engineering Principles**Modularity**

Each microservice handles a specific business function and can be maintained independently.

Scalability

Individual services can be scaled without scaling the entire application.

Maintainability

Refactoring, modular architecture, and automated testing make future changes easier.

Reliability

Testing, monitoring, version control, and rollback reduce the risk of failures.

Separation of Concerns

Different business functions are separated into suitable services.

Security

Security checks are integrated into development, testing, and deployment.

5. Justify the Chosen Methodology/Framework

5.1 Selected Methodology – Agile

Agile methodology is suitable for this project because legacy modernization is a continuous and incremental process. Instead of migrating the complete application at once, individual components can be migrated through multiple iterations.

5.2 Why Agile is Suitable

- Supports incremental development.
- Allows continuous testing.
- Provides regular stakeholder feedback.
- Handles changing requirements.
- Helps identify problems early.
- Reduces the risk of large-scale migration.
- Allows each migrated component to be reviewed before moving to the next.

Agile Development Process

Requirement Analysis



Sprint Planning



Development / Refactoring



Testing



Deployment



Monitoring



Feedback



Next Sprint

Agile therefore supports continuous modernization while maintaining business continuity.

6. Expected Outcomes and Limitations

6.1 Expected Benefits

The proposed platform is expected to:

1. Reduce technical debt.
2. Improve software maintainability.
3. Improve application scalability.
4. Reduce risks associated with migration.
5. Enable gradual modernization.
6. Improve deployment reliability.
7. Increase automated testing.
8. Improve application monitoring.
9. Reduce service disruption.
10. Provide measurable modernization progress.

6.2 How the Solution Addresses Challenges

Challenge	Solution
High technical debt	Technical-debt analysis
Tight coupling	Dependency analysis
Poor scalability	Microservices
Migration risk	Incremental migration
Testing difficulty	Automated testing
Manual deployment	CI/CD
Poor visibility	Monitoring
Migration failure	Rollback

6.3 Potential Risks

- Legacy behavior may be undocumented.
- Incorrect service boundaries may create additional complexity.
- Migration may temporarily affect performance.
- Data consistency may become difficult between services.
- Managing many microservices can increase operational complexity.
- Integration with old systems may require additional development.

Risk Mitigation

These risks can be reduced through incremental migration, automated regression testing, continuous monitoring, version control, and rollback mechanisms.

6.4 Limitations

- Complete modernization may take considerable time.
- Some legacy components may not be suitable for migration.
- Migration requires skilled developers and infrastructure.
- Microservices may increase operational complexity.
- Automated analysis may not fully understand business-specific logic.

6.5 Future Enhancements

Future versions can include:

- AI-based legacy code analysis.
- Automated migration recommendations.
- AI-assisted code refactoring.
- Automated API generation.
- Advanced dependency visualization.
- Predictive performance analysis.
- Cloud-native migration support.
- Automated modernization reports.
- Self-healing deployment and recovery.

Conclusion

The Legacy-to-Microservices Modernization and Technical-Debt Recovery Platform provides a systematic approach for modernizing legacy applications without completely replacing them. It combines technical-debt analysis, dependency analysis, API-based migration, microservices, containerization, automated testing, CI/CD, monitoring, and rollback.

The use of Agile methodology allows the system to be modernized incrementally while continuously testing and evaluating each change. By applying software engineering principles such as modularity, scalability, maintainability, reliability, security, and separation of concerns, the platform can reduce technical debt and improve the quality of the software system.

Overall, the proposed platform helps organizations modernize legacy applications safely, reduce maintenance difficulties, improve scalability and reliability, and achieve measurable modernization progress while maintaining business continuity.

Rubrics for Calculation

Criteria	Excellent (5 Marks)	Good (3 Marks)	Satisfactory (2 Marks)	Needs Improvement (1 Mark)
Problem Understanding (30%)	Clearly explains the problem, objectives, and scope with complete understanding.	Explains the problem with minor omissions.	Basic understanding with limited explanation.	Poor understanding; objectives and scope are unclear or missing.
Stakeholder & Challenge Analysis (25%)	Identifies all stakeholders, their roles, and challenges with clear analysis.	Identifies most stakeholders and challenges with adequate analysis.	Identifies only a few stakeholders or challenges.	Stakeholders and system analysis are incomplete or inaccurate.
Current System Analysis (20%)	Thoroughly analyzes the existing system, strengths, weaknesses, and identifies all gaps.	Good analysis with most strengths and weaknesses identified.	Limited analysis with partial identification of issues.	Proposed solution is unclear or inappropriate; requirements are largely missing.
Proposed Software Solution & Software Engineering Principles (15%)	Proposes an innovative, feasible solution applying appropriate software engineering principles (modularity, scalability, maintainability, security, etc.).	Suitable solution with good application of software engineering principles.	Basic solution with limited application of software engineering principles.	Methodology is inappropriate, poorly justified, or not discussed.
Methodology/ Framework Justification (10%)	Selects the most suitable SDLC/model/framework and provides strong technical justification.	Suitable methodology selected with adequate justification.	Methodology selected but justification is weak.	Outcomes, limitations, or future enhancements are missing; report lacks organization and clarity.

Criteria	Mark
Problem Understanding (30%)	/5
Stakeholder & Challenge Analysis (25%)	/5
Current System Analysis (20%)	/5
Proposed Software Solution & Software Engineering Principles (15%)	/5
Methodology/ Framework Justification (10%)	/5
TOTAL	/25